COMPILER DESIGN

Viva Questions & Answers

Simple, exam-friendly revision notes covering compiler basics, phases, tokens and patterns, Lex, YACC, parsers, NFA/DFA, and types of grammar.

TOPICS INCLUDED	QUICK REVISION
Compiler phases and symbol table	Short definitions
Lex, YACC, tokens and lexemes	Differences and examples
Parser types, LL(1), FIRST and FOLLOW	Common viva questions
NFA, DFA and grammar hierarchy	Easy-to-remember answers

Viva tip: Learn the definitions first, then practise the differences (Lex vs YACC, token vs lexeme, NFA vs DFA, and top-down vs bottom-up parsing).

1. Compiler – Basic Questions

1. What is a compiler?

Answer: A compiler is system software that translates a high-level programming language into machine code or another target language.

2. What is the main purpose of a compiler?

Answer: To translate source code into target code and report errors.

3. What is the difference between a compiler and an interpreter?

Answer: A compiler generally translates a program before execution; an interpreter translates and executes program instructions during execution.

4. What is source code?

Answer: The program written by the programmer in a programming language.

5. What is target code?

Answer: The output code produced by a compiler.

6. What are the main phases of a compiler?

Answer: Lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. Symbol-table management and error handling support these phases.

7. What is a symbol table?

Answer: A data structure that stores information about identifiers, such as their names, types, and scope.

8. What are common types of errors?

Answer: Lexical, syntax, and semantic errors are compiler-related errors. Runtime and logical errors are usually found during execution or testing.

2. Phases of a Compiler

1. What is lexical analysis?

Answer: The first phase of a compiler. It converts the input character stream into tokens.

2. What is syntax analysis?

Answer: It checks whether the tokens follow the grammar rules of the programming language.

3. What is semantic analysis?

Answer: It checks the meaning of the program, including type compatibility and valid declarations.

4. What is intermediate code generation?

Answer: It converts the program into an intermediate representation, such as three-address code.

5. What is code optimization?

Answer: It improves code efficiency without changing the program's meaning.

6. What is code generation?

Answer: It converts the intermediate representation into target code, such as assembly or machine code.

7. Which phase produces tokens?

Answer: Lexical analysis.

8. Which phase produces a parse tree?

Answer: Syntax analysis.

9. Which phase checks data types?

Answer: Semantic analysis.

10. What is the output of lexical analysis?

Answer: A sequence of tokens.

3. Patterns, Tokens and Lexemes

1. **What is a token?**
   Answer: A token is a category of lexical units, such as an identifier, keyword, operator, or constant.
2. **What is a lexeme?**
   Answer: The actual sequence of characters in the source program that matches a token.
3. **What is a pattern?**
   Answer: A rule that describes the form of lexemes belonging to a token.
4. **Give an example of token, lexeme and pattern.**
   Answer: For `int age = 20;`, `age` is a lexeme belonging to the identifier token category. A pattern describes valid identifiers.
5. **What are common types of tokens?**
   Answer: Keywords (`int`, `if`), identifiers (`age`), operators (`+`, `=`), constants (`20`), and separators (`;`, `()`).
6. **What is a regular expression?**
   Answer: A notation used to describe patterns of strings.
7. **Give a common identifier pattern.**
   Answer: `[A-Za-z_][A-Za-z0-9_]*` — it starts with a letter or underscore, followed by zero or more letters, digits, or underscores.

4. Lex

1. **What is Lex?**
   Answer: Lex is a tool that generates a lexical analyzer from regular-expression patterns and associated actions.
2. **What is the purpose of Lex?**
   Answer: To recognize tokens in source code.
3. **What are the three sections of a Lex program?**
   Answer: Definitions, rules, and user subroutines, separated by `%%`.
4. **What is the extension of a Lex file?**
   Answer: `.l`
5. **What is `yylex()`?**
   Answer: The generated function that performs lexical analysis and recognizes tokens.
6. **What is `yytext`?**
   Answer: It contains the text of the currently matched lexeme.
7. **What is `yyleng`?**
   Answer: It contains the length of the matched lexeme.
8. **What is `yywrap()`?**
   Answer: It tells the lexer what to do when it reaches the end of its input; traditionally, returning 1 indicates scanning is finished.
9. **What does `ECHO` do in Lex?**
   Answer: It prints the matched text to the output.
10. **What is the difference between Lex and a lexical analyzer?**
    Answer: A lexical analyzer is the component that recognizes tokens. Lex is a tool used to generate that component.

5. YACC

1. **What is YACC?**
   Answer: YACC stands for Yet Another Compiler Compiler. It generates a parser from a grammar specification.
2. **What is the purpose of YACC?**
   Answer: To perform syntax analysis according to grammar rules.

3. What is the extension of a YACC file?

Answer: `.y`

4. What are the three sections of a YACC program?

Answer: Declarations, grammar rules, and user subroutines, separated by `%%`.

5. What is `yyparse()`?

Answer: The generated function that performs syntax analysis.

6. What is `yyerror()`?

Answer: A function used to report syntax errors.

7. What is `yylval`?

Answer: A variable used to pass semantic values from the lexical analyzer to the parser.

8. How are Lex and YACC connected?

Answer: Lex recognizes tokens and returns them to YACC. YACC checks whether those tokens follow the grammar.

9. What is the difference between Lex and YACC?

Answer: Lex uses regular-expression patterns to recognize tokens and generates `yylex()`. YACC uses grammar rules to check syntax and generates `yyparse()`.

6. Parser and Types of Parser

1. What is a parser?

Answer: A component of a compiler that checks the syntactic structure of tokens according to grammar rules.

2. What is parsing?

Answer: The process of analyzing tokens to determine their grammatical structure.

3. What are the main types of parsers?

Answer: Top-down parsers and bottom-up parsers.

4. What is top-down parsing?

Answer: It starts from the start symbol and attempts to derive the input string.

5. What is bottom-up parsing?

Answer: It starts from the input string and reduces it to the start symbol.

6. What is recursive descent parsing?

Answer: A top-down parsing method in which recursive procedures implement grammar rules.

7. What is predictive parsing?

Answer: A top-down parsing technique that chooses productions using lookahead tokens without backtracking.

8. What is an LL(1) parser?

Answer: A top-down parser that scans input from Left to right, constructs a Leftmost derivation, and uses 1 lookahead token.

9. Why is it called LL(1)?

Answer: First L: left-to-right scanning; second L: leftmost derivation; 1: one lookahead symbol.

10. What is an LR parser?

Answer: A bottom-up parser that scans input from left to right and constructs a rightmost derivation in reverse.

11. What is shift-reduce parsing?

Answer: A bottom-up technique using operations such as shift, reduce, accept, and error.

12. What is left recursion?

Answer: A grammar is left-recursive when a nonterminal can derive a string beginning with itself. Example: `E → E + T | T`.

13. Why is left recursion a problem for recursive descent parsing?

Answer: It can cause infinite recursion.

14. What is left factoring?

Answer: A grammar transformation that removes common prefixes. Example: `A → ab | ac` becomes `A → aA'` and `A' → b | c`.

7. NFA and DFA

1. What is a finite automaton?

Answer: A mathematical model used to recognize patterns and regular languages.

2. What is NFA?

Answer: NFA stands for Nondeterministic Finite Automaton.

3. What is DFA?

Answer: DFA stands for Deterministic Finite Automaton.

4. What is the difference between NFA and DFA?

Answer: A DFA has exactly one transition for each state and input symbol. An NFA may have multiple possible transitions and may allow ε-transitions. Both recognize regular languages.

5. What is an ε-transition?

Answer: A transition that changes state without consuming an input symbol.

6. Can every NFA be converted into a DFA?

Answer: Yes. Subset construction converts an NFA into an equivalent DFA.

7. What is subset construction?

Answer: A method that represents sets of NFA states as states of a DFA.

8. What is an accepting state?

Answer: A state that indicates successful recognition when the input is consumed and the automaton ends in that state.

9. What is the relationship between regular expressions, NFA and DFA?

Answer: Regular expressions describe regular languages, and equivalent NFAs and DFAs can be built to recognize them.

10. Which automaton is used in lexical analysis?

Answer: Finite automata, especially DFA-based implementations, are commonly used to recognize tokens.

8. Types of Grammar

1. What is a grammar?

Answer: A set of production rules used to describe the structure of a language.

2. What are the four types in the Chomsky hierarchy?

Answer: Type 0: unrestricted; Type 1: context-sensitive; Type 2: context-free; Type 3: regular grammar.

3. What is a regular grammar?

Answer: A grammar with restricted production forms, typically right-linear or left-linear rules.

4. What is a context-free grammar (CFG)?

Answer: A grammar in which each production has a single nonterminal on its left-hand side. Example: `S → aSb | ε`.

5. What is a context-sensitive grammar?

Answer: A grammar whose productions generally do not decrease string length, subject to standard restrictions.

6. What is an unrestricted grammar?

Answer: A grammar with essentially unrestricted production rules, where the left-hand side contains at least one nonterminal.

7. Which grammar is used in syntax analysis?

Answer: Context-free grammar (CFG).

8. Which grammar is used to describe tokens?

Answer: Regular grammars and regular expressions.

9. What is a terminal symbol?

Answer: A symbol that cannot be expanded further by grammar productions, such as `a`, `+`, or `id`.

10. What is a nonterminal symbol?

Answer: A symbol that can be expanded using grammar productions, such as `E`, `T`, or `S`.

11. What is a production rule?

Answer: A rule defining how a nonterminal can be replaced by other symbols. Example: `E → E + T`.

12. What is a derivation?

Answer: The process of applying production rules to generate a string.

9. Extra Important Viva Questions

1. What is FIRST?

Answer: FIRST of a grammar symbol is the set of terminals that can appear first in strings derived from that symbol. It may include ε.

2. What is FOLLOW?

Answer: FOLLOW of a nonterminal is the set of terminals that can appear immediately after it in a sentential form. The end marker `$` belongs to FOLLOW of the start symbol.

3. What is an ambiguous grammar?

Answer: A grammar is ambiguous if at least one string has more than one distinct parse tree.

4. What is a parse tree?

Answer: A tree that represents how a string is derived from the grammar.

5. What is an abstract syntax tree (AST)?

Answer: A simplified tree representation of the essential structure of a program, omitting some grammatical details.

6. What is a handle in bottom-up parsing?

Answer: A substring matching the right-hand side of a production that can be reduced in one step of the reverse of a rightmost derivation.

7. What is the end marker used in parsing?

Answer: `$` commonly represents the end of input.

8. What is error recovery in parsing?

Answer: A method that allows the parser to continue after detecting a syntax error. Panic-mode recovery is a common technique.

9. What is the difference between syntax and semantics?

Answer: Syntax checks whether a program follows grammar rules; semantics checks whether the program has a valid meaning.

Quick Revision

Topic	Remember
Compiler	Translates source code into target code.
Lex	Recognizes tokens using patterns.
YACC	Generates a parser from grammar rules.
NFA → DFA	Use subset construction.
Parser types	Top-down and bottom-up.
Grammar hierarchy	Type 0 → Type 1 → Type 2 → Type 3, from unrestricted to regular.

Study priority: compiler phases; token vs. lexeme vs. pattern; Lex vs. YACC; LL(1); recursive descent; FIRST and FOLLOW; NFA-to-DFA conversion; and the four grammar types.